

REPORTE DE PRÁCTICA

NFC/RFID + BASE DE DATOS + APP

TECNOLOGIAS INALAMBRICAS

Fecha: 15/04/2026

Integrantes:

- Eduardo Cadengo López
- Itzel Citlalli Martell De La Cruz
- Damian Alexander Diaz Piña

Índice

1. Datos Generales	3
2. Descripción de la Tecnología RFID/NFC.....	4
3. Usos Actuales de la Tecnología	4
4. Funcionamiento y Descripción del Módulo PN532	5
Principio de operación	5
Características principales	5
5. Diagrama de Conexión (ESP32 – PN532 por I2C).....	5
Tabla de pines	5
Diagrama esquemático (ASCII)	6
6. Código Arduino IDE — Funcionamiento y Salidas Numéricas.....	7
Funciones de conversión numérica (líneas clave).....	7
Código completo (Arduino IDE)	7
7. Base de Datos MySQL con Laragon	10
Estructura de la base de datos	10
Script SQL de creación	10
8. Código Python lo importante — Aplicación de Escritorio (Tkinter + MySQL).....	17
Aspectos clave del código.....	17
Sección 1 — Configuración de conexión y constantes.....	17
Sección 2 — Conexión a la base de datos.....	18
Sección 3 — Lectura continua del puerto COM	18
Sección 4 — Validación del UID contra la base de datos.....	18
Sección 5 — Registro de eventos de acceso	19
9. Conclusiones	20
10. Videos, GitHub y Evidencias	20

1. Datos Generales

Práctica	Lectura de UID con tarjeta NFC y validación en base de datos
Plataforma	ESP32 (Tensilica Xtensa LX6)
Módulo sensor	NFC PN532 — comunicación I2C (SDA/SCL)
Entorno	Arduino IDE 2.x + librería Adafruit_PN532
Base de datos	MySQL 8 mediante Laragon (localhost)
Lenguaje externo	Python 3 — Tkinter (GUI) + pyserial + mysql-connector-python
Objetivo	Leer el UID de una tarjeta NFC/RFID, mostrarlo en el Monitor Serial en bases 2, 10 y 16, y validarlo contra una base de datos MySQL; la aplicación Python muestra la información del usuario o deniega el acceso según corresponda.

2. Descripción de la Tecnología RFID/NFC

RFID (Radio Frequency Identification) es una tecnología de identificación inalámbrica que utiliza radiofrecuencia para leer o escribir información en una etiqueta (tag) mediante un lector. En alta frecuencia (HF), opera a 13.56 MHz y se usa ampliamente para tarjetas de proximidad y sistemas de identificación.

NFC (Near Field Communication) puede considerarse una variante especializada de RFID HF, enfocada a distancias muy cortas (menores a 10 cm). Trabaja sobre estándares como ISO/IEC 14443 (tarjetas de proximidad), donde el lector genera el campo electromagnético y la tarjeta pasiva responde con su identificador único (UID).

Las tarjetas NFC/RFID almacenan un número de serie único (UID) de 4 o 7 bytes. Este UID es la base para los sistemas de control de acceso: el lector lo captura y un microcontrolador lo compara contra una lista autorizada —en esta práctica almacenada en MySQL— para conceder o denegar el acceso.

3. Usos Actuales de la Tecnología

- Control de acceso a edificios, laboratorios u oficinas mediante credenciales personales.
- Pagos sin contacto y validación rápida en puntos de venta (tarjetas bancarias NFC).
- Tarjetas de transporte público: validación de entrada y salida en autobuses y metros.
- Identificación, trazabilidad e inventarios en bibliotecas, almacenes y logística.
- Inicio y paro de maquinaria industrial: solo operadores autorizados pueden arrancar equipos.
- Autenticación en dispositivos móviles y llaves de hotel inteligentes.
- Gestión hospitalaria: identificación de pacientes, medicamentos y equipos médicos.
- Control de asistencia escolar o laboral mediante credenciales personales vinculadas a una base de datos.

4. Funcionamiento y Descripción del Módulo PN532

El PN532 es un controlador NFC integrado de NXP Semiconductors, diseñado para comunicación sin contacto a 13.56 MHz. Soporta los modos de lector/escritor para tarjetas ISO/IEC 14443A (MIFARE Classic, MIFARE Ultralight, NTAG) y puede comunicarse con el microcontrolador mediante tres interfaces: I2C, SPI o UART. En esta práctica se emplea el modo I2C (SDA/SCL).

Principio de operación

- Campo RF: el módulo genera un campo electromagnético a 13.56 MHz.
- Energización: cuando una tarjeta pasiva entra al campo, se energiza por inducción.
- Detección/Selección: el PN532 realiza el protocolo ISO14443A (REQA → ATQA → anticollision → SELECT).
- Entrega del UID: el número único de la tarjeta se transfiere al ESP32 mediante I2C para su validación.

Características principales

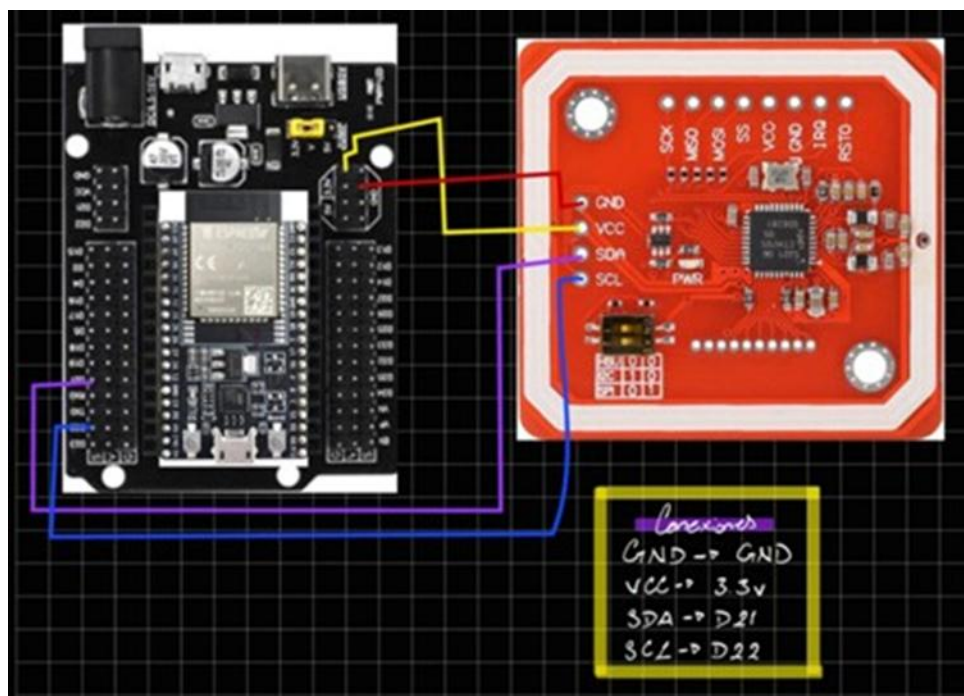
- Voltaje de operación: 3.3 V (algunos módulos incluyen regulador a 5 V).
- Frecuencia: 13.56 MHz (HF, alta frecuencia).
- Interfaces soportadas: I2C (esta práctica), SPI, UART/HSU.
- Soporta tarjetas ISO14443A/B y FeliCa; también modo Peer-to-Peer (NFC Forum).
- Librería Arduino: Adafruit_PN532 (disponible en el Gestor de Librerías del IDE).
- Distancia de lectura: hasta 5 cm en condiciones óptimas.

5. Diagrama de Conexión (ESP32 – PN532 por I2C)

Tabla de pines

ESP32	PN532	Descripción
3V3	VCC	Alimentación 3.3 V
GND	GND	Tierra común
GPIO 21 (SDA)	SDA	Datos I2C
GPIO 22 (SCL)	SCL	Reloj I2C

Diagrama esquemático (ASCII)



Nota: El protocolo I2C utiliza únicamente dos líneas: SDA (datos) y SCL (reloj). La librería Adafruit_PN532 gestiona automáticamente el protocolo de inicialización y lectura. No se requieren resistencias de pull-up externas si el módulo las incluye integradas.

6. Código Arduino IDE — Funcionamiento y Salidas Numéricas

El programa gestiona la lectura del UID de la tarjeta mediante el módulo PN532 a través de I2C y lo presenta en el Monitor Serial en tres representaciones numéricas distintas (base 2, 10 y 16), además de una versión concatenada utilizada como clave en la base de datos.

Funciones de conversión numérica (líneas clave)

Función / Instrucción	Base de salida	Descripción
imprimirBinario(uid[i])	Base 2 (BIN)	Recorre los 8 bits del byte de mayor a menor peso con desplazamiento >> i y máscara & 1.
Serial.print(uid[i], DEC)	Base 10 (DEC)	Arduino convierte el byte entero sin signo a su representación decimal.
nfc.PrintHex(uid, uidLength)	Base 16 (HEX)	Función de Adafruit que imprime cada byte como dos dígitos hexadecimales con prefijo 0x.
Serial.print(uid[i], HEX)	Base 16 (HEX)	Alternativa manual: imprime el byte en hex; antepone '0' si el valor es < 0x10.

Código completo (Arduino IDE)

```
/*
 * PROYECTO: CONTROL DE ACCESO en base de datos CON NFC/RFID USANDO ESP32 Y MÓDULO
PN532
 * Conexiones: PN532 SDA - GPIO 21 y PN532 SCL - GPIO 22
 */

#include <Wire.h>
#include <Adafruit_PN532.h>

// Definir pines I2C para ESP32
#define SDA_PIN 21
#define SCL_PIN 22

// Inicializar el objeto nfc usando I2C
Adafruit_PN532 nfc(SDA_PIN, SCL_PIN);

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n=====");
  Serial.println("SISTEMA NFC - LIBRERIA ADAFRUIT");
}
```

```

Serial.println("=====");

nfc.begin();

uint32_t versiondata = nfc.getFirmwareVersion();
if (!versiondata) {
    Serial.print("No se encontró el módulo PN532");
    while (1); // Detener si no hay módulo
}

// Configurar para leer tags RF
nfc.SAMConfig();
Serial.println("Esperando tarjeta...");
}

void loop() {
    uint8_t success;
    uint8_t uid[] = { 0, 0, 0, 0, 0, 0, 0 }; // Buffer para el UID
    uint8_t uidLength;                       // Longitud del UID (4 o 7 bytes)

    // Intentar leer una tarjeta
    success = nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &uidLength);

    if (success) {
        Serial.println("\n===== TARJETA DETECTADA =====");

        // 1. HEXADECIMAL
        Serial.print("UID HEX: ");
        nfc.PrintHex(uid, uidLength);

        // 2. DECIMAL
        Serial.print("UID DECIMAL: ");
        for (uint8_t i = 0; i < uidLength; i++) {
            Serial.print(uid[i], DEC);
            if (i < uidLength - 1) Serial.print(":");
        }
        Serial.println();

        // 3. BINARIO
        Serial.print("UID BINARIO: ");
        for (uint8_t i = 0; i < uidLength; i++) {
            imprimirBinario(uid[i]);
            if (i < uidLength - 1) Serial.print(":");
        }
        Serial.println();
    }
}

```



```
// 4. CONCATENADO (Para tu Base de Datos)
Serial.print("UID CONCATENADO: ");
for (uint8_t i = 0; i < uidLength; i++) {
    if (uid[i] < 0x10) Serial.print("0");
    Serial.print(uid[i], HEX);
}
Serial.println("\n=====");

delay(1000); // Evitar lecturas repetidas rápidas
}
}

// Función para imprimir en binario
void imprimirBinario(uint8_t valor) {
    for (int i = 7; i >= 0; i--) {
        Serial.print((valor >> i) & 1);
    }
}
```

7. Base de Datos MySQL con Laragon

Para esta práctica se utiliza Laragon como entorno de desarrollo local que proporciona un servidor MySQL sin necesidad de configuración adicional. La base de datos `nfc_control_acceso` contiene tres tablas principales que permiten registrar usuarios, tarjetas autorizadas y el historial de accesos.

Estructura de la base de datos

Tabla	Campos principales	Propósito
usuarios	id_usuario, nombre, apellido, email, telefono, ruta_imagen, estado	Datos del titular de la tarjeta
tarjetas_nfc	id_tarjeta, id_usuario (FK), uid_hexadecimal, estado	Tarjetas registradas y su estado
registro_acceso	id_registro, id_usuario (FK), uid_leido, tipo_acceso, estado_validacion, fecha_hora	Historial de todos los eventos

Script SQL de creación

```
-- -----
-- Host:                               127.0.0.1
-- Versión del servidor:                8.4.3 - MySQL Community Server - GPL
-- SO del servidor:                    Win64
-- HeidiSQL Versión:                   12.8.0.6908
-- -----

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET NAMES utf8 */;
/*!50503 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

-- Volcando estructura de base de datos para nfc_control_acceso
CREATE DATABASE IF NOT EXISTS `nfc_control_acceso` /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci */ /*!80016 DEFAULT ENCRYPTION='N' */;
USE `nfc_control_acceso`;

-- Volcando estructura para función nfc_control_acceso.obtener_usuario_por_uid
DELIMITER //
CREATE FUNCTION `obtener_usuario_por_uid`(uid_hex VARCHAR(50)) RETURNS int
READS SQL DATA
```

```

    DETERMINISTIC
BEGIN
    DECLARE id_user INT;

    SELECT id_usuario INTO id_user
    FROM tarjetas_nfc
    WHERE uid_hexadecimal = uid_hex AND estado = 'activa'
    LIMIT 1;

    RETURN IFNULL(id_user, 0);
END//
DELIMITER ;

-- Volcando estructura para procedimiento nfc_control_acceso.registrar_acceso
DELIMITER //
CREATE PROCEDURE `registrar_acceso`(
    IN p_uid_leido VARCHAR(50),
    IN p_tipo_acceso VARCHAR(20)
)
BEGIN
    DECLARE v_id_usuario INT;
    DECLARE v_id_tarjeta INT;
    DECLARE v_estado_validacion VARCHAR(50);

    -- Obtener usuario y tarjeta por UID
    SELECT id_usuario, id_tarjeta INTO v_id_usuario, v_id_tarjeta
    FROM tarjetas_nfc
    WHERE uid_hexadecimal = p_uid_leido AND estado = 'activa'
    LIMIT 1;

    -- Determinar estado de validación
    IF v_id_usuario IS NOT NULL THEN
        SET v_estado_validacion = 'exitoso';
    ELSE
        SET v_estado_validacion = 'no_registrado';
    END IF;

    -- Insertar en registro de acceso
    INSERT INTO registro_acceso (
        id_usuario,
        id_tarjeta,
        uid_leido,
        tipo_acceso,
        estado_validacion
    ) VALUES (

```

```

        v_id_usuario,
        v_id_tarjeta,
        p_uid_leido,
        p_tipo_acceso,
        v_estado_validacion
    );
END//
DELIMITER ;

-- Volcando estructura para tabla nfc_control_acceso.registro_acceso
CREATE TABLE IF NOT EXISTS `registro_acceso` (
  `id_acceso` int NOT NULL AUTO_INCREMENT,
  `id_usuario` int DEFAULT NULL,
  `id_tarjeta` int DEFAULT NULL,
  `uid_leido` varchar(50) DEFAULT NULL,
  `tipo_acceso` enum('entrada','salida','intento_fallido') DEFAULT 'entrada',
  `estado_validacion` enum('exitoso','fallido','no_registrado') DEFAULT 'exitoso',
  `fecha_hora` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  `detalles` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`id_acceso`),
  KEY `id_tarjeta` (`id_tarjeta`),
  KEY `idx_fecha_hora` (`fecha_hora`),
  KEY `idx_usuario` (`id_usuario`),
  KEY `idx_uid_leido` (`uid_leido`),
  KEY `idx_validacion` (`estado_validacion`),
  CONSTRAINT `registro_acceso_ibfk_1` FOREIGN KEY (`id_usuario`) REFERENCES
`usuarios` (`id_usuario`) ON DELETE SET NULL,
  CONSTRAINT `registro_acceso_ibfk_2` FOREIGN KEY (`id_tarjeta`) REFERENCES
`tarjetas_nfc` (`id_tarjeta`) ON DELETE SET NULL
) ENGINE=InnoDB AUTO_INCREMENT=91 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci;

-- Volcando datos para la tabla nfc_control_acceso.registro_acceso: ~32 rows
(aproximadamente)
DELETE FROM `registro_acceso`;
INSERT INTO `registro_acceso` (`id_acceso`, `id_usuario`, `id_tarjeta`, `uid_leido`,
`tipo_acceso`, `estado_validacion`, `fecha_hora`, `detalles`) VALUES
(1, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:36:13', NULL),
(2, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:36:22', NULL),
(3, NULL, NULL, '491C3307', 'entrada', 'no_registrado', '2026-04-09 17:36:25',
NULL),
(4, NULL, NULL, '491C3307', 'entrada', 'no_registrado', '2026-04-09 17:36:27',
NULL),
(5, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:36:35', NULL),

```

```

(6, NULL, NULL, '491C3307', 'entrada', 'no_registrado', '2026-04-09 17:36:46',
NULL),
(7, NULL, NULL, '491C3307', 'entrada', 'no_registrado', '2026-04-09 17:36:48',
NULL),
(8, NULL, NULL, '491C3307', 'entrada', 'no_registrado', '2026-04-09 17:36:58',
NULL),
(9, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:37:00', NULL),
(10, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:42:16', NULL),
(11, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:42:28', NULL),
(12, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:42:29', NULL),
(13, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:42:30', NULL),
(14, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:42:32', NULL),
(15, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:42:33', NULL),
(16, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 17:42:35', NULL),
(17, 4, NULL, '0D1B1207', 'entrada', 'exitoso', '2026-04-09 18:07:28', NULL);

-- Volcando estructura para tabla nfc_control_acceso.roles
CREATE TABLE IF NOT EXISTS `roles` (
  `id_rol` int NOT NULL AUTO_INCREMENT,
  `nombre_rol` varchar(50) NOT NULL,
  `descripcion` varchar(200) DEFAULT NULL,
  `permisos_json` json DEFAULT NULL,
  PRIMARY KEY (`id_rol`),
  UNIQUE KEY `nombre_rol` (`nombre_rol`)
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Volcando datos para la tabla nfc_control_acceso.roles: ~3 rows (aproximadamente)
DELETE FROM `roles`;
INSERT INTO `roles` (`id_rol`, `nombre_rol`, `descripcion`, `permisos_json`) VALUES
(1, 'Administrador', 'Acceso total al sistema', NULL),
(2, 'Usuario Normal', 'Acceso básico', NULL),
(3, 'Visitante', 'Acceso limitado temporal', NULL);

-- Volcando estructura para tabla nfc_control_acceso.tarjetas_nfc
CREATE TABLE IF NOT EXISTS `tarjetas_nfc` (
  `id_tarjeta` int NOT NULL AUTO_INCREMENT,
  `id_usuario` int NOT NULL,
  `uid_hexadecimal` varchar(50) NOT NULL,
  `uid_decimal` varchar(50) DEFAULT NULL,
  `uid_binario` varchar(200) DEFAULT NULL,
  `descripcion` varchar(200) DEFAULT NULL,
  `fecha_registro` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  `fecha_vencimiento` date DEFAULT NULL,
  `estado` enum('activa','inactiva') DEFAULT 'activa',
  PRIMARY KEY (`id_tarjeta`),

```

```

    UNIQUE KEY `uid_hexadecimal` (`uid_hexadecimal`),
    KEY `idx_uid_hex` (`uid_hexadecimal`),
    KEY `idx_usuario` (`id_usuario`),
    KEY `idx_estado_tarjeta` (`estado`),
    CONSTRAINT `tarjetas_nfc_ibfk_1` FOREIGN KEY (`id_usuario`) REFERENCES `usuarios`
(`id_usuario`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=7 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Volcando datos para la tabla nfc_control_acceso.tarjetas_nfc: ~4 rows
(aproximadamente)
DELETE FROM `tarjetas_nfc`;
INSERT INTO `tarjetas_nfc` (`id_tarjeta`, `id_usuario`, `uid_hexadecimal`,
`uid_decimal`, `uid_binario`, `descripcion`, `fecha_registro`, `fecha_vencimiento`,
`estado`) VALUES
(1, 1, '3AB42DF1', '58:180:45:241', NULL, 'Tarjeta NFC de Juan García - Acceso
principal', '2026-04-09 14:57:22', NULL, 'activa'),
(2, 1, 'A7F8C2E4', '167:248:194:228', NULL, 'Tarjeta NFC de Juan García - Acceso
secundario', '2026-04-09 14:57:22', NULL, 'activa'),
(3, 2, '5C1D9B6E', '92:29:155:110', NULL, 'Tarjeta NFC de María López', '2026-04-
09 14:57:22', NULL, 'activa'),
(4, 3, '8E4A7D2C', '142:74:125:44', NULL, 'Tarjeta NFC de Carlos Martínez', '2026-
04-09 14:57:22', NULL, 'activa'),
(5, 4, '0D1B1207', '13:27:18:7', '00001101:00011011:00010010:00000111', 'Tarjeta
NFC de Axel Yoab', '2026-04-09 15:39:06', '2026-04-09', 'activa'),
(6, 5, '491C3307', '73:28:51:7', '01001001:00011100:00110011:00000111', 'Tarjeta
NFC de DamianTron', '2026-04-15 17:27:10', '2026-04-15', 'activa');

-- Volcando estructura para tabla nfc_control_acceso.usuarios
CREATE TABLE IF NOT EXISTS `usuarios` (
  `id_usuario` int NOT NULL AUTO_INCREMENT,
  `nombre` varchar(100) NOT NULL,
  `apellido` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `telefono` varchar(20) DEFAULT NULL,
  `ruta_imagen` varchar(255) DEFAULT NULL,
  `estado` enum('activo','inactivo') DEFAULT 'activo',
  `fecha_registro` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id_usuario`),
  UNIQUE KEY `email` (`email`),
  KEY `idx_email` (`email`),
  KEY `idx_estado_usuario` (`estado`),
  FULLTEXT KEY `idx_nombre_apellido` (`nombre`,`apellido`)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

```

-- Volcando datos para la tabla nfc_control_acceso.usuarios: ~4 rows
(aproximadamente)
DELETE FROM `usuarios`;
INSERT INTO `usuarios` (`id_usuario`, `nombre`, `apellido`, `email`, `telefono`,
`ruta_imagen`, `estado`, `fecha_registro`) VALUES
  (1, 'Juan', 'García', 'juan.garcia@empresa.com', '5551234567', NULL, 'activo',
'2026-04-09 14:57:22'),
  (2, 'María', 'López', 'maria.lopez@empresa.com', '5559876543', NULL, 'activo',
'2026-04-09 14:57:22'),
  (3, 'Carlos', 'Martínez', 'carlos.martinez@empresa.com', '5552468135', NULL,
'activo', '2026-04-09 14:57:22'),
  (4, 'Axel', 'Yoab', 'Axel@gmail.com', '4491234556',
'C:\\\\Users\\\\Lalo\\\\Downloads\\\\Inalambricas\\\\PracticasNFC\\\\didpod.jpeg',
'activo', '2026-04-09 15:37:51'),
  (5, 'Damian', 'Piñata', 'Dami@gmail.com', '4491234566',
'C:\\\\Users\\\\Lalo\\\\Downloads\\\\Inalambricas\\\\PracticasNFC\\\\damianpod.jpeg',
'inactivo', '2026-04-15 17:22:12');

-- Volcando estructura para tabla nfc_control_acceso.usuario_roles
CREATE TABLE IF NOT EXISTS `usuario_roles` (
  `id_usuario` int NOT NULL,
  `id_rol` int NOT NULL,
  `fecha_asignacion` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id_usuario`,`id_rol`),
  KEY `id_rol` (`id_rol`),
  CONSTRAINT `usuario_roles_ibfk_1` FOREIGN KEY (`id_usuario`) REFERENCES `usuarios`
(`id_usuario`) ON DELETE CASCADE,
  CONSTRAINT `usuario_roles_ibfk_2` FOREIGN KEY (`id_rol`) REFERENCES `roles`
(`id_rol`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Volcando datos para la tabla nfc_control_acceso.usuario_roles: ~3 rows
(aproximadamente)
DELETE FROM `usuario_roles`;
INSERT INTO `usuario_roles` (`id_usuario`, `id_rol`, `fecha_asignacion`) VALUES
  (1, 1, '2026-04-09 14:57:22'),
  (2, 2, '2026-04-09 14:57:22'),
  (3, 2, '2026-04-09 14:57:22');

-- Volcando estructura para función nfc_control_acceso.validar_uid
DELIMITER //
CREATE FUNCTION `validar_uid`(uid_hex VARCHAR(50)) RETURNS int
  READS SQL DATA
  DETERMINISTIC
BEGIN

```

```

DECLARE resultado INT;

SELECT COUNT(*) INTO resultado
FROM tarjetas_nfc
WHERE uid_hexadecimal = uid_hex AND estado = 'activa';

RETURN resultado;
END//
DELIMITER ;

-- Volcando estructura para vista nfc_control_acceso.vista_acceso_usuarios
-- Creando tabla temporal para superar errores de dependencia de VIEW
CREATE TABLE `vista_acceso_usuarios` (
  `id_acceso` INT NOT NULL,
  `nombre` VARCHAR(1) NULL COLLATE 'utf8mb4_0900_ai_ci',
  `apellido` VARCHAR(1) NULL COLLATE 'utf8mb4_0900_ai_ci',
  `email` VARCHAR(1) NULL COLLATE 'utf8mb4_0900_ai_ci',
  `uid_hexadecimal` VARCHAR(1) NULL COLLATE 'utf8mb4_0900_ai_ci',
  `tipo_acceso` ENUM('entrada','salida','intento_fallido') NULL COLLATE
'utf8mb4_0900_ai_ci',
  `estado_validacion` ENUM('exitoso','fallido','no_registrado') NULL COLLATE
'utf8mb4_0900_ai_ci',
  `fecha_hora` TIMESTAMP NULL,
  `nombre_rol` VARCHAR(1) NULL COLLATE 'utf8mb4_0900_ai_ci'
) ENGINE=MyISAM;

-- Eliminando tabla temporal y crear estructura final de VIEW
DROP TABLE IF EXISTS `vista_acceso_usuarios`;
CREATE ALGORITHM=UNDEFINED SQL SECURITY DEFINER VIEW `vista_acceso_usuarios` AS
select `ra`.`id_acceso` AS `id_acceso`,`u`.`nombre` AS `nombre`,`u`.`apellido` AS
`apellido`,`u`.`email` AS `email`,`tn`.`uid_hexadecimal` AS
`uid_hexadecimal`,`ra`.`tipo_acceso` AS `tipo_acceso`,`ra`.`estado_validacion` AS
`estado_validacion`,`ra`.`fecha_hora` AS `fecha_hora`,`r`.`nombre_rol` AS
`nombre_rol` from ((((`registro_acceso` `ra` left join `usuarios` `u`
on((`ra`.`id_usuario` = `u`.`id_usuario`))) left join `tarjetas_nfc` `tn`
on((`ra`.`id_tarjeta` = `tn`.`id_tarjeta`))) left join `usuario_rol` `ur`
on((`u`.`id_usuario` = `ur`.`id_usuario`))) left join `roles` `r` on((`ur`.`id_rol`
= `r`.`id_rol`)))) order by `ra`.`fecha_hora` desc;

/*!40103 SET TIME_ZONE=IFNULL(@OLD_TIME_ZONE, 'system') */;
/*!40101 SET SQL_MODE=IFNULL(@OLD_SQL_MODE, '') */;
/*!40014 SET FOREIGN_KEY_CHECKS=IFNULL(@OLD_FOREIGN_KEY_CHECKS, 1) */;
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40111 SET SQL_NOTES=IFNULL(@OLD_SQL_NOTES, 1) */;

```


8. Código Python lo Importante — Aplicación de Escritorio (Tkinter + MySQL)

Todo el código estará aparte, aquí solo pondremos lo mas importante ya que es demasiado código y solo amontonaría más el documento, el archivo es: `app_nfc_control.py` y se encontrara en el RAR o en el GitHub

La aplicación de escritorio desarrollada en Python integra tres componentes principales: la interfaz gráfica construida con Tkinter, la lectura del puerto COM mediante pyserial (datos provenientes de la ESP32) y la validación/registro de accesos en la base de datos MySQL a través de mysql-connector-python.

Por la extensión del código, se presentan únicamente las secciones más relevantes y educativas. El archivo completo (`app_nfc_control.py`) está disponible en el repositorio GitHub del proyecto.

Aspectos clave del código

- Configuración de la conexión a MySQL mediante un diccionario `DB_CONFIG`.
- Lectura del puerto COM en un hilo separado (threading) para no bloquear la interfaz gráfica.
- Validación del UID mediante consulta SQL JOIN entre `tarjetas_nfc` y `usuarios`.
- Registro de cada evento de acceso (exitoso o denegado) en la tabla `registro_acceso`.
- Actualización dinámica de la interfaz Tkinter desde el hilo de lectura usando `root.after()`.

Sección 1 — Configuración de conexión y constantes

// — IMPORTS Y CONFIGURACIÓN INICIAL —

```
import tkinter as tk    // GUI de escritorio
from tkinter import ttk, messagebox
import serial           // Comunicación con puerto COM (pyserial)
import serial.tools.list_ports    // Listar puertos disponibles
import threading        // Hilo separado para lectura serial
import mysql.connector  // Conector MySQL para Python
from mysql.connector import Error
from datetime import datetime
from PIL import Image, ImageTk    // Mostrar imágenes de usuarios (Pillow)
import json
```

```
DB_CONFIG = {    // Parámetros de conexión a Laragon/MySQL
    'host':      'localhost',
    'user':      'root',
    'password':  '',    // Sin contraseña por defecto en Laragon
    'database':  'nfc_control_acceso',
    'port':      3306
}
```

```
SERIAL_CONFIG = {    // Parámetros del puerto serie
```

```
'baudrate': 115200,    // Debe coincidir con Serial.begin() en ESP32
'timeout': 1
}
```

Sección 2 — Conexión a la base de datos

```
// — MÉTODO conectar_base_datos —
def conectar_base_datos(self):
    try:
        self.conexion_bd = mysql.connector.connect(**DB_CONFIG)    // Desempaqueta dict
                             como argumentos
        if self.conexion_bd.is_connected():
            self.label_estado_bd.config(
                text='Estado BD: Conectado',
                foreground='green')
    except Error as e:
        messagebox.showerror('Error BD', str(e))
```

Sección 3 — Lectura continua del puerto COM

```
// — MÉTODO leer_puerto_serial - Hilo secundario —
def leer_puerto_serial(self):
    ser = serial.Serial(    // Abre el puerto con los parámetros configurados
        port=puerto_seleccionado,
        **SERIAL_CONFIG)
    while self.ejecutando:
        raw = ser.readline()    // Bloquea hasta recibir \n o cumplir timeout
        if not raw:
            continue
        linea = raw.decode('utf-8', errors='replace').strip()
        if linea.startswith('{'):    // Filtra tramas JSON provenientes de la ESP32
            datos = json.loads(linea)
            uid = datos.get('hex', '')
            self.root.after(0,    // Actualiza GUI desde hilo secundario de forma segura
                lambda u=uid: self.actualizar_ui(u))
```

Sección 4 — Validación del UID contra la base de datos

```
// — MÉTODO procesar_uid_leido —
def procesar_uid_leido(self, uid):
    cursor = self.conexion_bd.cursor(dictionary=True)
    consulta = '''
        SELECT u.*, tn.id_tarjeta    // JOIN para obtener datos del usuario dueño de la
tarjeta
        FROM usuarios u
        JOIN tarjetas_nfc tn ON u.id_usuario = tn.id_usuario
        WHERE tn.uid_hexadecimal = %s    // Parámetro evita inyección SQL
```

```

        AND tn.estado = 'activa'
    LIMIT 1
'''
cursor.execute(consulta, (uid,))
usuario = cursor.fetchone()

if usuario:    // UID encontrado → acceso exitoso
    self.mostrar_informacion_usuario(usuario)
    self.registrar_acceso(uid, usuario['id_usuario'], 'exitoso')
else:    // UID no registrado → acceso denegado
    self.mostrar_acceso_denegado(uid)
    self.registrar_acceso(uid, None, 'no_registrado')

```

Sección 5 — Registro de eventos de acceso

```

// — MÉTODO registrar_acceso —
def registrar_acceso(self, uid, id_usuario, estado):
    cursor = self.conexion_bd.cursor()
    consulta = '''
        INSERT INTO registro_acceso
        (id_usuario, uid_leido, tipo_acceso, estado_validacion)
        VALUES (%s, %s, %s, %s)
    '''

    cursor.execute(consulta,    // Inserta el evento con sus parámetros
        (id_usuario, uid, 'entrada', estado))
    self.conexion_bd.commit()    // Confirma la transacción en la BD
    self.cargar_accesos_recientes()    // Refresca la tabla de historial en la GUI

```

9. Conclusiones

Eduardo:

Esta práctica permitió comprender de manera integral cómo funciona la tecnología NFC/RFID en un sistema real de control de acceso con base de datos. La integración entre la ESP32 y el módulo PN532 resultó confiable para la obtención del UID en los tres formatos numéricos. La incorporación de MySQL mediante Laragon demostró que es posible construir sistemas de acceso escalables con datos persistentes, y la aplicación Python con Tkinter permitió visualizar en tiempo real la respuesta del sistema con retroalimentación visual al usuario.

Itzel:

La práctica evidenció cómo una tecnología cotidiana puede implementarse desde cero a nivel de hardware y software. El diseño de la base de datos con Laragon y la conexión desde Python resultaron más intuitivos de lo esperado, y el uso de consultas JOIN permite relacionar tarjetas con usuarios de manera eficiente. La visualización del UID en diferentes bases numéricas refuerza conceptos de representación de datos, mientras que la interfaz gráfica hace accesible el sistema a usuarios sin conocimientos técnicos.

Damian:

La adición de una base de datos MySQL al sistema NFC elevó la complejidad y el realismo de la práctica. El sistema completo desde la lectura del UID en la ESP32, la transmisión serial, la validación en la BD y el registro del evento demuestra un flujo de datos coherente y escalable. La posibilidad de ampliar el sistema con más tarjetas, usuarios o puntos de acceso sin modificar el hardware confirma la flexibilidad de la arquitectura implementada.

10. Videos, GitHub y Evidencias

GitHub del proyecto:

https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/NFC_RFID_COM_APP_BD

Video de demostración:

https://youtube.com/shorts/i_WNIqsTOLc

Evidencias:

